

INTERVIEW PREPARATION

WellDev Trainee Software Engineer

MCQ Practice Bank — 210 Questions with Answers & Explanations

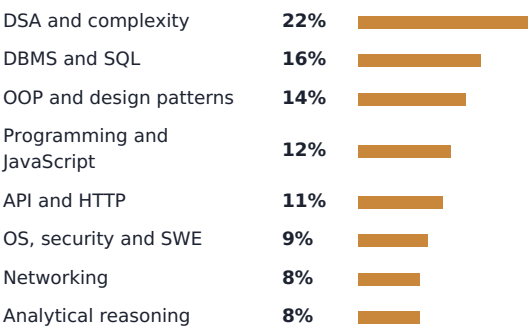
Paper 1 — Full Mock Paper · 100 questions · Sections A-H
Paper 2 — Core Topic Drill · 110 questions · Sections A-F
Appendix — Night-before facts & revision checklist
Coverage — DSA · DBMS/SQL · OOP · Design Patterns
OS · Networking · HTTP/REST · JavaScript · Aptitude

Original predicted question bank — not an official or leaked WellDev paper.
Every answer is marked inline directly beneath its question.

The answer and a short explanation sit in a green panel directly under each question. To sit this as a timed mock, cover the green panels with a sheet of paper, work through a whole section, then uncover and mark. Anything you get wrong should go straight onto the checklist at the back.

Reported screening format. Candidate reports from July 2026 describe the WellDev screening round as a challenging, fundamentals-heavy MCQ paper. Reported topic areas include DBMS, operating systems, design patterns, APIs, DSA, recursion, SQL and problem solving. Earlier reports additionally mention JavaScript, networking, subnetting, HTTP status codes, minimum spanning trees, sorting, greedy algorithms and analytical mathematics. Notes marked “asked in Round 1” flag items reported by previous candidates — treat those as top priority.

Estimated weight of each area (use this to allocate revision time):



Suggested three-day plan. Day 1 — work Paper 2 section by section (it is organised by topic, so gaps show up as clusters). Day 2 — sit Paper 1 as a closed-book 100-minute mock, then mark and review only the sections you scored below 70% on. Day 3 — re-read the Appendix, redo every question you missed, and stop.

Suggested time: 100 minutes. Cover the green answer panels before you start.

Section A — Programming Fundamentals and JavaScript

Questions 1-12

1. In most C expressions, the name of an array evaluates to:

- A. The complete array
- B. A pointer to its first element
- C. The size of the array
- D. The last element

Answer: B — a pointer to its first element. This is **array-to-pointer decay**: `arr` becomes `&arr[0]` of type `int*`. The three exceptions where no decay happens are `sizeof arr`, `&arr`, and a string literal used to initialise a char array.

2. Given `int arr[3] = {1, 2, 3};` which statement is correct about `&arr[0]` and `&arr`?

- A. Their numerical addresses are the same, but their pointer types are different
- B. Their types and values are identical
- C. Their addresses are different
- D. `&arr` is invalid

Answer: A. `&arr[0]` has type `int*`; `&arr` has type `int (*) [3]`. They start at the same byte, but pointer arithmetic differs: `&arr[0] + 1` moves by `sizeof(int)` = 4 bytes, while `&arr + 1` moves by `sizeof(arr)` = 12 bytes. **Asked in a previous Round 1.**

3. What is the output?

```
int i = 5;
cout << i++ << endl;
cout << ++i << endl;
```

- A. 5 6
- B. 5 7
- C. 6 7
- D. 6 6

Answer: B — 5 then 7. Post-increment yields the old value (prints **5**) and leaves `i` = 6. Pre-increment increments first, so `i` becomes 7 and **7** is printed.

4. What is the time complexity of accessing `arr[500]` in an array?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Answer: A — $O(1)$. Array elements are contiguous, so the address is computed arithmetically: `address = base + index × element_size`. The index does not affect the cost.

5. What is the time complexity of accessing the k^{th} node of a singly linked list?

- A. $O(1)$
- B. $O(\log k)$
- C. $O(k)$
- D. $O(k^2)$

Answer: C — $O(k)$. Nodes are scattered in memory and linked only by pointers, so you must walk from the head following `k` next-pointers. **The array-vs-linked-list pair (Q4 + Q5) was asked in Round 1.**

6. What are the time and auxiliary-space complexities of this function?

```
def sum_n(n):
    if n == 0:
        return 0
    return n + sum_n(n - 1)
```

- A. Time $O(1)$, space $O(1)$
- B. Time $O(n)$, space $O(1)$
- C. Time $O(n)$, space $O(n)$
- D. Time $O(n^2)$, space $O(n)$

Answer: C — time $O(n)$, space $O(n)$. `n` nested calls are alive at once, so `n` stack frames are held. Python does not perform tail-call optimisation, so the depth is real (and hits the recursion limit near `n = 1000`).

7. An iterative implementation of the preceding function normally has:

- A. Time $O(n)$, space $O(1)$
- B. Time $O(1)$, space $O(n)$
- C. Time $O(\log n)$, space $O(1)$
- D. Time $O(n^2)$, space $O(1)$

Answer: A — time $O(n)$, space $O(1)$. Same number of additions, but a single accumulator replaces the call stack. **Round 1 asked exactly this recursion-versus-iteration comparison** — the point is that time is unchanged and only space differs.

8. In the expression `false && expensiveFunction()`, will `expensiveFunction()` execute?

- A. Always
- B. Never
- C. Only if it returns true
- D. It causes an error

Answer: B — never. **Short-circuit evaluation:** with `&&`, a false left operand already determines the result, so the right side is skipped. Same idea with `||` and a true left operand — which is why `if (p != NULL && p->x)` is safe.

9. What does `typeof null` return in JavaScript?

- A. "null"
- B. "undefined"
- C. "object"
- D. "boolean"

Answer: C — "object". A long-standing bug from the first JavaScript implementation, kept forever for backwards compatibility. To test for null use `value === null`.

10. What is the primary difference between == and === in JavaScript?

- A. === performs type conversion
- B. == never performs type conversion
- C. === compares without implicit type coercion
- D. They are always identical

Answer: C. === is strict: it compares type first, then value. == coerces, which produces the classic surprises — 0 == "0" is true, 0 == false is true, null == undefined is true. Prefer ===.

11. A variable declared using let is:

- A. Function-scoped only
- B. Block-scoped
- C. Globally scoped
- D. Class-scoped only

Answer: B — block-scoped. let and const are confined to the nearest { }, cannot be redeclared in the same scope, and sit in the temporal dead zone until initialised. var is function-scoped and hoisted as undefined.

12. What is the output order?

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
```

- A. A B C D
- B. A D B C
- C. A D C B
- D. A C D B

Answer: C — A D C B. Synchronous code runs first (**A, D**). Then the event loop drains the **microtask** queue, where promise callbacks live (**C**). Macrotasks such as setTimeout run last (**B**), even with a 0 ms delay. Microtasks always beat macrotasks.

Section B — OOP and Design Patterns

Questions 13–26

13. Which OOP concept bundles data and methods while restricting direct access?

- A. Inheritance
- B. Encapsulation
- C. Polymorphism
- D. Aggregation

Answer: B — encapsulation. Textbook form: private fields reached only through public getters and setters, so the class controls every change to its own state.

14. Abstraction primarily means:

- A. Copying one class into another
- B. Hiding implementation details and exposing essential behaviour
- C. Making every field public
- D. Creating multiple constructors

Answer: B. The usual one-line distinction: **abstraction hides complexity at design level** (what an object does), **encapsulation hides and binds data at implementation level** (how it protects state).

15. Inheritance generally represents which relationship?

- A. Has-a
- B. Uses-a
- C. Is-a
- D. Depends-on

Answer: C — is-a. A Dog *is a* Animal. Composition gives has-a: a Car *has an* Engine. Modern guidance is to favour composition over inheritance.

16. Runtime polymorphism is commonly implemented using:

- A. Method overriding
- B. Method overloading
- C. Constructor chaining
- D. Static variables

Answer: A — method overriding. Also called dynamic or late binding: the actual method is chosen from the object's real type at run time, typically through a virtual function table.

17. Method overloading is usually considered:

- A. Runtime polymorphism
- B. Compile-time polymorphism
- C. Encapsulation
- D. Dependency injection

Answer: B — compile-time (static) polymorphism. The compiler picks the right overload from the argument types in the signature. Return type alone cannot distinguish two overloads.

18. Which statement describes method overriding?

- A. Same method name with different parameter lists in one class
- B. A subclass provides a new implementation of an inherited method
- C. A private method is called twice
- D. Two classes have unrelated methods with the same name

Answer: B. Option A describes overloading — the standard distractor. Overriding requires the same name *and* the same signature, plus an inheritance (is-a) relationship.

19. Which statement about an abstract class is normally correct?

- A. It can always be instantiated directly
- B. It cannot contain implemented methods
- C. It cannot normally be instantiated directly
- D. It cannot contain fields

Answer: C. It *can* have constructors (invoked via super()), fields and fully implemented concrete methods — it simply cannot be created on its own with new.

20. A protected member is generally accessible from:

- A. Every class
- B. Only the operating system
- C. The declaring class and its subclasses
- D. No class

Answer: C. In Java specifically, protected also covers the whole package, and the difference from package-private (default) is exactly the case of a **subclass in a different package**. Asked in a previous Round 1.

21. Composition represents which relationship?

- A. Is-a
- B. Has-a
- C. Same-as
- D. Overrides-a

Answer: B — has-a. Composition implies ownership and shared lifetime (a House has Rooms; destroy the house and the rooms go too). Aggregation is the weaker has-a where the part outlives the whole.

22. The Single Responsibility Principle states that a class should:

- A. Have only one method
- B. Have one reason to change
- C. Have only one object
- D. Inherit from only one class

Answer: B — one reason to change. The S in SOLID. "One method" and "one object" (Singleton) are the traps.

23. Dependency Injection means:

- A. A class creates every dependency internally
- B. Dependencies are supplied to an object from outside
- C. Database records are injected into SQL
- D. A subclass removes its parent dependency

Answer: B. Delivered by constructor, setter or interface injection. It is a form of **Inversion of Control** and the practical way to realise the Dependency Inversion Principle; it is what makes classes testable with mocks. **WellDev asked "what do you understand by Dependency Injection?" in Round 1.**

24. The Singleton pattern is intended to:

- A. Create multiple interchangeable algorithms
- B. Ensure a class has one instance and provide access to it
- C. Add behaviour dynamically
- D. Convert one interface into another

Answer: B. Creational pattern. Implementation: **private constructor + static accessor** holding the single instance. Know the thread-safety follow-up (double-checked locking, or an eagerly initialised static field).

25. Which pattern is appropriate for constructing a complex object step by step?

- A. Builder
- B. Observer
- C. Adapter
- D. Proxy

Answer: A — Builder. Creational. It removes telescoping constructors and lets the same build process produce different representations — think a fluent chain such as `new Pizza.Builder().size(12).cheese().build()`.

26. Which pattern encapsulates interchangeable algorithms behind a common interface?

- A. Singleton
- B. Strategy
- C. Decorator
- D. Facade

Answer: B — Strategy. Behavioural. The client selects the algorithm at run time — different payment methods, different sorting or compression algorithms, all behind one interface.

Section C — Data Structures and Algorithms

Questions 27–48

27. A stack follows:

- A. FIFO
- B. LIFO
- C. Random order
- D. Priority order

Answer: B — LIFO. Last in, first out. Push, pop and peek are all $O(1)$. Used for call stacks, undo, and expression parsing.

28. A normal queue follows:

- A. FIFO
- B. LIFO
- C. Sorted order
- D. Reverse order

Answer: A — FIFO. Enqueue at the rear, dequeue from the front, both $O(1)$ with a linked list or circular buffer.

29. Can a stack be implemented using queues?

- A. No
- B. Yes, using one or more queues
- C. Only using a linked list
- D. Only if the queue is sorted

Answer: B — yes. Classic exercise. With two queues, make either push or pop the expensive $O(n)$ operation. It also works with a single queue: after each push, rotate the earlier elements to the back.

30. Inserting an element at the beginning of an array normally takes:

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n \log n)$

Answer: C — $O(n)$. Every existing element must shift one position right to open the slot. Appending at the end is $O(1)$ amortised.

31. Inserting a node at the head of a singly linked list takes:

- | | |
|-----------|----------------|
| A. $O(1)$ | B. $O(\log n)$ |
| C. $O(n)$ | D. $O(n^2)$ |

Answer: A — $O(1)$. Point the new node's next at the old head and move the head pointer. Nothing shifts. This is the mirror image of Q30 and the reason linked lists exist.

32. Binary search on a sorted array has a worst-case time complexity of:

- | | |
|-----------|------------------|
| A. $O(1)$ | B. $O(\log n)$ |
| C. $O(n)$ | D. $O(n \log n)$ |

Answer: B — $O(\log n)$. Each comparison discards half the remaining range, so at most $\lceil \log_2 n \rceil$ steps. It requires random access *and* sorted data.

33. Why does binary search usually not provide its normal benefit on a linked list?

- A. Linked lists cannot be sorted
B. A linked list cannot store integers
C. Accessing the middle node is not $O(1)$
D. Linked lists do not support comparisons

Answer: C. You can still apply the comparisons, but reaching each midpoint costs a linear walk, so the total collapses to $O(n)$ — no better than a plain scan.

34. Which data structure is normally used by BFS?

- | | |
|----------|---------------|
| A. Stack | B. Queue |
| C. Heap | D. Hash table |

Answer: B — queue. FIFO produces level-by-level exploration, which is why BFS finds the shortest path in an unweighted graph.

35. Which structure is normally used by iterative DFS?

- | | |
|----------|-----------|
| A. Queue | B. Stack |
| C. Trie | D. B-tree |

Answer: B — stack. An explicit stack when iterative; the recursive version uses the call stack, which is the same thing. **DFS/BFS structures were asked in Round 1 — memorise the pair: DFS = stack, BFS = queue.**

36. Which traversal prints a Binary Search Tree in descending order?

- | | |
|--------------------|--------------------|
| A. Left-Root-Right | B. Root-Left-Right |
| C. Right-Root-Left | D. Left-Right-Root |

Answer: C — Right-Root-Left (reverse inorder). Plain inorder (L-Root-R) gives ascending order, so mirroring it gives descending. **Round 1 asked how to iterate a BST in reverse order.**

37. In a full binary tree, every node has:

- | | |
|---------------------------|--------------------------------|
| A. Exactly one child | B. Either zero or two children |
| C. At most three children | D. A left child only |

Answer: B — zero or two children. No node has exactly one child. Do not confuse *full* with *complete* (Q38) or *perfect* (every level entirely filled).

38. A complete binary tree has:

- A. Every level completely full without exception
B. All levels full except possibly the last, which is filled from left to right
C. Only leaf nodes
D. Every node with exactly two children

Answer: B. This shape is exactly why a binary heap can live in a plain array with no wasted slots: children of index i sit at $2i+1$ and $2i+2$.

39. The best-case time complexity of insertion sort is:

- | | |
|-----------|----------------|
| A. $O(1)$ | B. $O(\log n)$ |
| C. $O(n)$ | D. $O(n^2)$ |

Answer: C — $O(n)$. On already-sorted input each element needs one comparison and no shifts. This is why insertion sort is the best simple choice for nearly sorted data, and why hybrid sorts fall back to it on small runs.

40. Selection sort normally performs how many comparisons?

- | | |
|------------------|-------------|
| A. $O(\log n)$ | B. $O(n)$ |
| C. $O(n \log n)$ | D. $O(n^2)$ |

Answer: D — $O(n^2)$. Always $n(n-1)/2$ comparisons regardless of input — best, average and worst are all $O(n^2)$. Its one virtue is the minimum number of **swaps**: $n-1$. **Selection vs insertion sort was asked in Round 1.**

41. The worst-case time complexity of merge sort is:

- | | |
|------------------|----------------|
| A. $O(n)$ | B. $O(\log n)$ |
| C. $O(n \log n)$ | D. $O(n^2)$ |

Answer: C — $O(n \log n)$. Guaranteed in every case, and it is stable — but it needs $O(n)$ auxiliary space, which is the usual reason quicksort is preferred in practice.

42. The worst-case time complexity of quicksort is:

- | | |
|------------------|-------------|
| A. $O(\log n)$ | B. $O(n)$ |
| C. $O(n \log n)$ | D. $O(n^2)$ |

Answer: D — $O(n^2)$. Happens when the pivot is consistently the smallest or largest element, giving maximally unbalanced partitions. Average case is $O(n \log n)$; randomised or median-of-three pivots make the worst case very unlikely.

43. What is the approximate time complexity of naïve recursive Fibonacci?

```
int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n - 1) + fib(n - 2);  
}
```

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. Exponential, commonly written $O(2^n)$

Answer: D — exponential, $O(2^n)$. Every call spawns two more and the same subproblems are recomputed endlessly. **Memoisation or bottom-up DP reduces it to $O(n)$ time.** Asked in Round 1.

44. Average-case lookup in a well-designed hash table is:

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Answer: A — $O(1)$ average. The worst case is $O(n)$, when every key collides into one bucket and the chain degenerates into a list. Good hash function + low load factor keeps it constant.

45. Which structure is particularly suitable for prefix-based string searching?

- A. Stack
- B. Trie
- C. Queue
- D. Circular linked list

Answer: B — trie (prefix tree). Lookup costs $O(L)$ in the key length, independent of how many words are stored. Powers autocomplete and spell-check. **Asked in Round 1.**

46. Which structure is commonly used to implement a priority queue?

- A. Binary heap
- B. Normal array without ordering
- C. Stack
- D. Trie

Answer: A — binary heap. $O(\log n)$ insert and extract-min, $O(1)$ peek. Underpins Dijkstra's algorithm, Prim's algorithm and Huffman coding.

47. Which algorithm detects a cycle in a linked list using constant extra space?

- A. Dijkstra's algorithm
- B. Floyd's tortoise-and-hare algorithm
- C. Prim's algorithm
- D. Kadane's algorithm

Answer: B. A slow pointer moves one step, a fast pointer two; if they meet, there is a cycle. $O(n)$ time, $O(1)$ space — a hash set would also work but costs $O(n)$ space.

48. Which algorithm finds a Minimum Spanning Tree?

- A. KMP
- B. Kruskal's algorithm
- C. Bellman-Ford only
- D. Binary search

Answer: B — Kruskal's (and Prim's). KMP is string matching; Bellman-Ford is single-source shortest path with negative weights. An MST of V vertices always has exactly $V-1$ edges.

Section D — DBMS and SQL

Questions 49–64

49. Which is the correct simplified logical SQL execution order?

- A. SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY
- B. FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
- C. WHERE → SELECT → FROM → ORDER BY → GROUP BY
- D. ORDER BY → FROM → SELECT → WHERE

Answer: B. Full order: **FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.** This explains why a SELECT alias cannot be used in WHERE (WHERE runs first) but can be used in ORDER BY. **This exact question appeared in Round 1.**

50. What is the main purpose of HAVING?

- A. Filter individual rows before grouping
- B. Filter groups after aggregation
- C. Rename columns
- D. Sort records

Answer: B. WHERE filters rows *before* GROUP BY and cannot contain aggregates; HAVING filters the grouped result and can.

51. Which ACID property means a transaction happens completely or not at all?

- A. Consistency
- B. Isolation
- C. Atomicity
- D. Durability

Answer: C — Atomicity. All-or-nothing. Consistency = valid state to valid state; Isolation = concurrent transactions do not interfere; **Durability = committed data survives a crash.**

52. Database normalization mainly helps to:

- A. Increase duplicated data
- B. Reduce redundancy and modification anomalies
- C. Remove all foreign keys
- D. Make every query constant time

Answer: B. It targets the three anomalies — insert, update and delete. The trade-off is more joins, which is precisely why denormalization is used to speed up reads.

53. First Normal Form requires:

- A. Every table to have two keys
- B. Atomic column values without repeating groups
- C. No foreign keys
- D. No nullable columns

Answer: B. One value per cell. A phone_numbers column holding "017..., 019..." breaks 1NF.

54. Second Normal Form removes:

- A. All primary keys
- B. Partial dependencies on part of a composite key
- C. Transitive dependencies only
- D. Every duplicate value

Answer: B. Relevant only when the key is composite: no non-key attribute may depend on just part of it.

55. Third Normal Form mainly removes:

- A. Transitive dependencies between non-key attributes
- B. Primary keys
- C. All indexes
- D. Every many-to-many relationship

Answer: A. No non-key attribute may depend on another non-key attribute ($A \rightarrow B \rightarrow C$). One-liner to memorise: *every non-key attribute depends on the key, the whole key, and nothing but the key.*

56. An INNER JOIN returns:

- A. Every row from both tables
- B. Only matching rows from both tables
- C. Only unmatched rows
- D. All rows from the left table

Answer: B. Rows with no counterpart on either side are dropped entirely.

57. Which join produces a Cartesian product?

- A. INNER JOIN
- B. LEFT JOIN
- C. CROSS JOIN
- D. NATURAL JOIN

Answer: C — CROSS JOIN. Every left row paired with every right row: $10 \times 5 = 50$ rows. It is also what you get accidentally from a comma join with no WHERE condition.

58. A LEFT JOIN returns:

- A. Only rows matching in both tables
- B. All rows from the left table and matching rows from the right
- C. All rows from the right table only
- D. Only unmatched left rows

Answer: B. Unmatched left rows survive with NULLs on the right, so a 10-row left table returns **at least 10 rows**. Adding WHERE right.id IS NULL gives you the "left only" anti-join.

59. Which query correctly identifies duplicate combinations of name and email?

- A. SELECT name, email FROM users WHERE COUNT(*) > 1;
- B. SELECT name, email, COUNT(*) FROM users GROUP BY name, email HAVING COUNT(*) > 1;
- C. SELECT DISTINCT name, email FROM users;
- D. DELETE DUPLICATE FROM users;

Answer: B. Group by the candidate duplicate columns and keep groups of size > 1. **A is invalid SQL** — aggregate functions cannot appear in WHERE. C removes duplicates instead of finding them, and D is not valid SQL at all.

60. Which query finds the second-highest distinct salary?

- A. SELECT MAX(salary) FROM employees;
- B. SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);
- C. SELECT MIN(salary) FROM employees;
- D. SELECT salary FROM employees ORDER BY salary;

Answer: B. The word **distinct** matters: this finds the highest salary strictly below the maximum, so ties at the top do not break it. It returns NULL if every salary is equal. Also know the window-function form: DENSE_RANK() OVER (ORDER BY salary DESC) = 2, and the ORDER BY salary DESC LIMIT 1 OFFSET 1 version.

61. Which database constraint prevents duplicate values in a column?

- A. DEFAULT
- B. CHECK
- C. UNIQUE
- D. FOREIGN KEY

Answer: C — UNIQUE. Unlike a PRIMARY KEY, a UNIQUE column can usually hold NULLs (and a table may have many UNIQUE constraints but only one primary key).

62. Which statement is correct?

- A. DROP removes rows but keeps the table structure
- B. TRUNCATE removes all rows but normally keeps the table structure
- C. DELETE always removes the table definition
- D. TRUNCATE supports a WHERE clause

Answer: B. Three-way summary: **DELETE** is DML, row-by-row, accepts WHERE, fires triggers, can be rolled back. **TRUNCATE** is DDL, deallocates pages, no WHERE, generally no row triggers, usually resets auto-increment. **DROP** removes the table definition entirely.

63. A database index generally:

- A. Speeds up every write without any cost
- B. Can improve reads but consume space and slow some writes
- C. Removes the need for primary keys
- D. Encrypts database records

Answer: B. Every INSERT, UPDATE and DELETE must also maintain each index. Related facts: a table has at most **one clustered index** (it defines physical row order), and a **hash index cannot serve range queries** — use a B+ tree for those.

64. Which replication model allows multiple nodes to update the same replicated dataset?

- A. Snapshot only
- B. Master-slave
- C. Multi-master
- D. Read-only replication

Answer: C — multi-master. Every node accepts writes, which buys availability at the price of write-conflict resolution. Master-slave (primary-replica) allows writes on one node only.

Section E — REST API, HTTP and Web

Questions 65–75

65. A RESTful server should normally treat each request as:

- A. Dependent on hidden server session state
- B. Self-contained and stateless
- C. A database transaction only
- D. A WebSocket message

Answer: B. Statelessness means the server keeps no client session between requests — each request carries all context it needs (for example a bearer token). That is what makes REST APIs horizontally scalable. It does *not* mean nothing is persisted.

66. Which HTTP method is normally safe and idempotent?

- A. GET
- B. POST
- C. CONNECT
- D. None

Answer: A — GET. Safe (read-only): GET, HEAD, OPTIONS. **Idempotent** (same result if repeated): GET, HEAD, OPTIONS, PUT, DELETE. POST and PATCH are neither.

67. Which statement best describes PUT and PATCH?

- A. PUT usually replaces a complete representation; PATCH applies a partial update
- B. PATCH deletes a resource
- C. PUT is only for reading
- D. They have exactly the same semantics

Answer: A. PUT is idempotent because sending the same full representation twice leaves the same state. PATCH is not necessarily idempotent (e.g. an "increment by 1" patch).

68. Which method is commonly used to create a subordinate resource?

- A. GET
- B. POST
- C. HEAD
- D. TRACE

Answer: B — POST. `POST /users` creates a new user under the collection. Repeating it creates another resource, which is exactly why POST is not idempotent.

69. Which status code normally means a resource was created successfully?

- A. 200
- B. 201
- C. 204
- D. 301

Answer: B — 201 Created. Should be accompanied by a Location header pointing at the new resource. 200 OK is the generic success.

70. Which status code means success with no response body?

- A. 201
- B. 204
- C. 304
- D. 404

Answer: B — 204 No Content. Typical response to a successful DELETE. 304 Not Modified is a caching response, not a success-with-no-body.

71. Which status code usually means the client sent an invalid request?

- A. 200
- B. 301
- C. 400
- D. 500

Answer: C — 400 Bad Request. Class map: 2xx success, 3xx redirect, **4xx client error**, **5xx server error**. 422 Unprocessable Entity is the common choice for validation failures specifically.

72. Which statement correctly differentiates 401 and 403?

- A. 401 means not authenticated; 403 means access is forbidden
- B. 401 means server failure; 403 means success
- C. 401 means not found; 403 means redirected
- D. They mean exactly the same thing

Answer: A. Memorable version: **401 = "I don't know who you are"** (missing or invalid credentials — logging in may fix it). **403 = "I know who you are and you still can't"** (authenticated but not authorised).

73. Which status code means the requested resource was not found?

- | | |
|--------|--------|
| A. 400 | B. 401 |
| C. 404 | D. 409 |

Answer: C — 404 Not Found. Sometimes returned deliberately instead of 403 to avoid revealing that a protected resource exists.

74. Which status code commonly represents a conflict with the current resource state?

- | | |
|--------|--------|
| A. 202 | B. 301 |
| C. 409 | D. 502 |

Answer: C — 409 Conflict. Typical uses: registering an email that already exists, or an edit that loses an optimistic-locking race.

75. What does HTTP 429 mean?

- | | |
|--------------------------|-----------------------|
| A. Unauthorized | B. Too Many Requests |
| C. Internal Server Error | D. Permanent Redirect |

Answer: B — 429 Too Many Requests. Rate limiting. Usually paired with a Retry-After header. Worth also knowing 500 Internal Server Error, 502 Bad Gateway and 503 Service Unavailable.

Section F — Networking

Questions 76–83

76. What is the purpose of a subnet mask?

- A. Encrypt network traffic
- B. Distinguish the network portion from the host portion of an IP address
- C. Identify the application port
- D. Convert domain names into IP addresses

Answer: B. The 1-bits mark the network prefix, the 0-bits mark host bits. /24 = 255.255.255.0, /26 = 255.255.255.192.

77. Which OSI layer performs logical addressing and routing?

- | | |
|-------------------|-----------------------|
| A. Physical layer | B. Data Link layer |
| C. Network layer | D. Presentation layer |

Answer: C — Network layer (Layer 3). IP addressing and routers live here. Switches are Layer 2 (MAC addresses); hubs are Layer 1.

78. TCP and UDP operate primarily at which OSI layer?

- | | |
|------------|----------------|
| A. Network | B. Transport |
| C. Session | D. Application |

Answer: B — Transport (Layer 4). Layer order 1–7: Physical, Data Link, Network, Transport, Session, Presentation, Application. Mnemonic: *Please Do Not Throw Sausage Pizza Away*.

79. DNS primarily converts:

- | | |
|---------------------------------|-----------------------------------|
| A. MAC addresses into passwords | B. Domain names into IP addresses |
| C. HTTP into HTTPS | D. Private IPs into subnet masks |

Answer: B. Runs on **UDP port 53** for ordinary queries, and TCP port 53 for zone transfers and oversized responses. ARP is the different one: it maps IP → MAC.

80. Which statement is correct?

- A. UDP guarantees ordered delivery
- B. TCP provides reliable, ordered delivery
- C. TCP has no connection setup
- D. UDP always retransmits lost packets

Answer: B. TCP is connection-oriented (three-way handshake SYN → SYN-ACK → ACK), with acknowledgements, retransmission, ordering and flow control. UDP is fire-and-forget with minimal overhead — better for live video and voice, where a retransmitted stale frame is useless.

81. How many usable host addresses are normally available in an IPv4 /26 subnet?

- | | |
|-------|-------|
| A. 26 | B. 32 |
| C. 62 | D. 64 |

Answer: C — 62. A /26 leaves 6 host bits: $2^6 = 64$ total addresses, minus the network address and the broadcast address = **62 usable**. General formula: $2^{(32-\text{prefix})} - 2$.

82. What is the network address of 192.168.1.70/26?

- | | |
|-----------------|------------------|
| A. 192.168.1.0 | B. 192.168.1.64 |
| C. 192.168.1.70 | D. 192.168.1.128 |

Answer: B — 192.168.1.64. Block size is 64, so the /26 subnets in the last octet are **0-63, 64-127, 128-191, 192-255**. 70 falls in the second block: network .64, broadcast .127, usable .65–.126. **Round 1 reportedly included "find the network address" — practise several of these.**

83. Under historical classful addressing, the default subnet mask for a Class B network is:

- | | |
|------------------|--------------------|
| A. 255.0.0.0 | B. 255.255.0.0 |
| C. 255.255.255.0 | D. 255.255.255.255 |

Answer: B — 255.255.0.0 (/16). Class A = /8, Class B = /16, Class C = /24. Private ranges to memorise: **10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16**.

Section G — OS, Security and Software Engineering

Questions 84–92

84. The operating-system kernel primarily:

- A. Designs web pages
- B. Manages hardware, processes, memory, and system resources
- C. Stores only user passwords
- D. Compiles JavaScript

Answer: B. It is the core of the OS, runs in privileged (kernel) mode, and is entered from user mode by a system call. **Asked almost verbatim in Round 1.**

85. Which statement about threads in the same process is normally correct?

- A. They cannot communicate
- B. They share process memory and resources
- C. Each thread requires a separate operating system
- D. Threads cannot run concurrently

Answer: B. They share the code segment, heap, globals and file descriptors, but each thread has its **own stack, registers and program counter**. That shared memory is exactly why race conditions need mutexes.

86. Which set contains the four necessary deadlock conditions?

- A. Mutual exclusion, hold and wait, no preemption, circular wait
- B. Caching, paging, swapping, scheduling
- C. Authentication, authorization, encryption, hashing
- D. Compilation, execution, testing, deployment

Answer: A. All four must hold simultaneously; breaking any one prevents deadlock. Banker's algorithm is deadlock **avoidance**, not prevention or detection.

87. Virtual memory allows:

- A. Programs to use an address space larger than available physical RAM
- B. Every instruction to run in constant time
- C. Networks to operate without routers
- D. Databases to avoid indexes

Answer: A. Pages move between RAM and disk on demand. Overdo it and you get **thrashing** — more time spent paging than executing.

88. Why is a unique salt added before password hashing?

- A. To make passwords shorter
- B. To ensure identical passwords do not automatically produce identical stored hashes
- C. To allow administrators to retrieve the password
- D. To replace the hashing algorithm

Answer: B. It defeats precomputed rainbow tables and forces an attacker to crack each account separately. Use a slow, memory-hard algorithm — **bcrypt, scrypt or Argon2** — never plain MD5 or SHA-256.

89. Which is the strongest standard defence against SQL injection?

- A. Concatenating user input into SQL
- B. Parameterized queries or prepared statements
- C. Hiding the database name
- D. Using longer table names

Answer: B. Parameter binding keeps user input as *data*, so it can never be reinterpreted as SQL code. Input validation and least-privilege DB accounts are useful extra layers, not substitutes.

90. Agile development generally emphasizes:

- A. One final delivery with no feedback
- B. Iterative delivery, collaboration, and responding to change
- C. Avoiding customer involvement
- D. Writing no documentation at all

Answer: B. The Agile Manifesto prefers working software and customer collaboration over rigid plans — it does not forbid documentation. WellDev describes itself as running an agile process, so expect this framing.

91. A unit test usually verifies:

- A. The complete production environment
- B. A small isolated unit of behaviour
- C. Only database backups
- D. The internet connection

Answer: B. Fast, isolated, dependencies mocked. Integration tests check components together; end-to-end tests exercise the whole system.

92. Continuous Integration generally means:

- A. Developers integrate changes frequently with automated builds and tests
- B. Production is never updated
- C. Every developer uses a separate programming language
- D. Tests are executed only once per year

Answer: A. Merge small changes often so integration problems surface immediately. Continuous **Delivery** then keeps every build releasable; continuous **Deployment** ships it automatically.

93. A jar contains five red and five white balls. What is the minimum number you must draw blindly to guarantee at least three balls of the same colour?

- A. 3
- B. 4
- C. 5
- D. 6

Answer: C — 5. Pigeonhole principle, worst case first: you could draw 2 red and 2 white (4 balls) with no triple. The 5th ball must be red or white, completing three of one colour. General form for k colours and m of a kind: $k(m-1) + 1$.

94. A shopkeeper receives a fake ₹1,000 note for a ₹300 product. He obtains genuine change from another shop, gives the customer ₹700 and the product, then later repays the other shop ₹1,000. What is his total loss?

- A. ₹300
- B. ₹700
- C. ₹1,000
- D. ₹1,300

Answer: C — ₹1,000. Track only what leaves his hands to the **customer**: ₹700 in change plus a product worth ₹300 = ₹1,000. The neighbouring shop is a wash — he received ₹1,000 of real money and repaid ₹1,000. The fake note itself is worth nothing, so it is not a separate loss. (If the question asked for the loss valued at cost price of the product, the figure would differ; here the product is valued at ₹300.)

95. For a 100-floor building and two eggs, what is the minimum worst-case number of drops needed using the optimal strategy?

- A. 10
- B. 12
- C. 14
- D. 20

Answer: C — 14. Use a decreasing step size so the worst case is flat. Start at floor 14, then 27, 39, 50, 60, 69, 77, 84, 90, 95, 99, 100. Solve $n(n+1)/2 \geq 100$: **14 × 15 / 2 = 105 ≥ 100**, while 13 gives only 91. If the first egg breaks at the k-th jump you linearly scan the small interval below it with the second egg, and the total is always ≤ 14.

96. Find the next number: 2, 6, 12, 20, 30, ?

- A. 36
- B. 40
- C. 42
- D. 44

Answer: C — 42. The terms are $n(n+1)$: 1·2, 2·3, 3·4, 4·5, 5·6, **6·7 = 42**. Equivalently the differences are 4, 6, 8, 10, **12**.

97. What is the probability of getting at least one head when a fair coin is tossed twice?

- A. 1/4
- B. 1/2
- C. 3/4
- D. 1

Answer: C — 3/4. Use the complement: $P(\text{no head}) = 1/2 \times 1/2 = 1/4$, so $P(\text{at least one}) = 1 - 1/4 = 3/4$. Sample space HH, HT, TH, TT — three of four qualify.

98. A completes a task in six days and B completes it in three days. Working together at constant rates, how long will they take?

- A. 1 day
- B. 2 days
- C. 3 days
- D. 4 days

Answer: B — 2 days. Add the rates, not the times: $1/6 + 1/3 = 1/2$ of the job per day, so the whole job takes 2 days. Shortcut for two workers: $(a \times b)/(a + b) = 18/9 = 2$.

99. A product is bought for ₹800 and sold for ₹920. What is the profit percentage?

- A. 12%
- B. 15%
- C. 20%
- D. 25%

Answer: B — 15%. Profit = $920 - 800 = 120$. Profit % is always taken on **cost price**: $120/800 = 0.15 = 15\%$.

100. All trainees are learners. Some learners are programmers. Which conclusion is guaranteed?

- A. All programmers are trainees
- B. Some trainees are programmers
- C. All trainees are learners
- D. No learner is a programmer

Answer: C. The only guaranteed conclusion is a restatement of a given premise. **B is the trap**: the programmer-learners could all sit in the part of "learners" that contains no trainees, so the overlap is possible but not guaranteed. In syllogism questions, "some" never upgrades to a certainty.

Organised by topic rather than mixed, so weak areas show up as clusters. Harder on average than Paper 1 — several items are deliberate trick questions.

Section A — DBMS and Normalization

22 questions

A1. R(A, B, C, D) has candidate key (A, B) and the FD $B \rightarrow C$. Which normal form is violated?

- a) 1NF
- b) 2NF
- c) 3NF
- d) BCNF

Answer: (b) 2NF. C depends on B, which is only **part** of the composite key — a textbook partial dependency.

A2. R(A, B, C) with $A \rightarrow B$ and $B \rightarrow C$, where A is the primary key. This violates:

- a) 2NF
- b) 3NF
- c) BCNF only
- d) 4NF

Answer: (b) 3NF. $A \rightarrow B \rightarrow C$ is a **transitive dependency**: the non-key attribute C depends on the non-key attribute B.

A3. Which statement is true?

- a) Every 3NF relation is in BCNF
- b) Every BCNF relation is in 3NF
- c) They are equivalent
- d) BCNF is weaker than 2NF

Answer: (b). BCNF is **stricter**, so $BCNF \Rightarrow 3NF$ but not the reverse. The normal forms nest: $BCNF \subset 3NF \subset 2NF \subset 1NF$.

A4. A relation is in BCNF if, for every non-trivial FD $X \rightarrow Y$:

- a) Y is a prime attribute
- b) X is a superkey
- c) X is a candidate key only
- d) Y is atomic

Answer: (b). Every determinant must be a superkey — the cleanest one-line definition of BCNF.

A5. Which decomposition is always both lossless and dependency-preserving?

- a) BCNF
- b) 3NF
- c) 4NF
- d) 5NF

Answer: (b) 3NF. Classic trick question. A BCNF decomposition guarantees lossless join but may *lose* a functional dependency; only 3NF guarantees both properties together.

A6. 1NF is violated when:

- a) There is a transitive dependency
- b) A column holds multi-valued or repeating groups
- c) There is a partial dependency
- d) There are two candidate keys

Answer: (b). 1NF is purely about atomic cell values.

A7. 4NF deals with:

- a) Functional dependency
- b) Multivalued dependency
- c) Join dependency
- d) Partial dependency

Answer: (b) multivalued dependency. 5NF (project-join normal form) is the one that handles **join** dependency.

A8. Denormalization is done primarily to:

- a) Save storage
- b) Remove anomalies
- c) Improve read performance by reducing joins
- d) Enforce referential integrity

Answer: (c). It deliberately reintroduces redundancy to trade write complexity and storage for faster reads — common in reporting and analytics tables.

A9. Correct logical order of SQL execution:

- a) SELECT $\rightarrow$ FROM $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ ORDER BY
- b) FROM $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ SELECT $\rightarrow$ ORDER BY
- c) FROM $\rightarrow$ SELECT $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ ORDER BY $\rightarrow$ HAVING
- d) WHERE $\rightarrow$ FROM $\rightarrow$ SELECT $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ ORDER BY

Answer: (b). Appeared in their Round 1. Full order: FROM/JOIN $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ SELECT $\rightarrow$ DISTINCT $\rightarrow$ ORDER BY $\rightarrow$ LIMIT.

A10. Why can't you use a column alias defined in SELECT inside the WHERE clause?

- a) Syntax restriction only
- b) WHERE executes before SELECT
- c) Aliases are session-scoped
- d) You can, in all RDBMS

Answer: (b). The alias does not exist yet when WHERE runs. ORDER BY, which runs after SELECT, *can* use it.

A11. WHERE vs HAVING:

- a) HAVING filters rows before grouping
- b) WHERE can contain aggregate functions
- c) HAVING filters groups after GROUP BY
- d) They are interchangeable

Answer: (c). Put a condition in WHERE whenever you can — filtering rows before grouping is cheaper than filtering groups afterwards.

A12. ACID — which property ensures committed data survives a crash?

- a) Atomicity
- b) Consistency
- c) Isolation
- d) Durability

Answer: (d) Durability. Implemented through write-ahead logging: the log record hits stable storage before the commit is acknowledged.

A13. Which isolation level still permits phantom reads?

- a) Read Uncommitted
- b) Read Committed
- c) Repeatable Read
- d) All of the above

Answer: (d) — but the point of the question is (c). Every level below Serializable allows phantoms. The intended discriminator: **Repeatable Read prevents dirty reads and non-repeatable reads but still allows phantoms**; only Serializable prevents all three.

A14. A "dirty read" means:

- a) Reading uncommitted data from another transaction
- b) Reading stale cached data
- c) Reading a locked row
- d) Reading NULL values

Answer: (a). Possible only at **Read Uncommitted**; if the other transaction rolls back, you have read data that never existed.

A15. How many clustered indexes can a table have?

- a) Unlimited
- b) One
- c) One per column
- d) Zero

Answer: (b) one. It defines the **physical order of the rows**, and rows can only be stored in one order. Non-clustered indexes are unlimited.

A16. Which index type cannot efficiently serve a range query such as WHERE age BETWEEN 20 AND 30?

- a) B+ tree
- b) Hash index
- c) Composite index
- d) Clustered index

Answer: (b) hash index. Hashing destroys ordering, so it only supports equality lookups. B+ trees keep keys sorted with linked leaves, which is exactly what a range scan needs.

A17. Adding many indexes to a table primarily hurts:

- a) SELECT
- b) INSERT / UPDATE / DELETE
- c) JOIN
- d) Nothing

Answer: (b). Every write must update every affected index, plus the extra storage cost.

A18. Table A has 10 rows, Table B has 5 rows. A LEFT JOIN B returns at least:

- a) 5
- b) 10
- c) 15
- d) 50

Answer: (b) 10. Every left row survives; unmatched ones get NULLs on the right. It can return *more* than 10 if one left row matches several right rows.

A19. CROSS JOIN of 10 × 5 rows produces:

- a) 15
- b) 50
- c) 10
- d) 5

Answer: (b) 50. Cartesian product: every pairing, $m \times n$.

A20. Which finds the second highest salary without LIMIT/TOP?

- a) SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp)
- b) SELECT salary FROM emp ORDER BY salary DESC
- c) SELECT MIN(salary) FROM emp
- d) SELECT DISTINCT salary FROM emp

Answer: (a). Returns NULL if all salaries are equal. Also know the window-function form: DENSE_RANK() OVER (ORDER BY salary DESC) = 2.

A21. Which shows duplicate rows?

- a) SELECT col FROM t GROUP BY col HAVING COUNT(*) > 1
- b) SELECT DISTINCT col FROM t
- c) SELECT col FROM t WHERE COUNT(*) > 1
- d) SELECT col FROM t ORDER BY col

Answer: (a). Option (c) is **invalid SQL** — aggregates cannot sit in WHERE.

A22. DELETE vs TRUNCATE:

- a) TRUNCATE is DML, DELETE is DDL
- b) DELETE fires row triggers, TRUNCATE generally doesn't
- c) TRUNCATE accepts a WHERE clause
- d) DELETE resets the auto-increment counter

Answer: (b). Option (a) has it backwards (DELETE is DML, TRUNCATE is DDL); (c) is false; and it is **TRUNCATE**, not DELETE, that normally resets the auto-increment counter.

Section B — Object-Oriented Programming

16 questions

B1. The four pillars of OOP are:

- a) Encapsulation, Abstraction, Inheritance, Polymorphism
- b) Class, Object, Method, Field
- c) Coupling, Cohesion, Binding, Scope
- d) Overloading, Overriding, Hiding, Shadowing

Answer: (a). Be ready to define each in one sentence — this is a near-certain interview follow-up.

B2. Method overloading is resolved at:

- a) Runtime
- b) Compile time
- c) Link time
- d) Load time

Answer: (b) compile time. Static / early binding. Overriding is the opposite: runtime / dynamic / late binding.

B3. Overriding requires:

- a) Same name, different parameter list
- b) Same name and same signature, plus an IS-A relationship
- c) Different return type only
- d) Static methods

Answer: (b). The overriding method also cannot reduce visibility, and static methods are *hidden*, not overridden.

B4. In Java, a protected member is accessible:

- a) Only inside the class
- b) Only inside the package
- c) Inside the package and in subclasses outside the package
- d) Everywhere

Answer: (c). The difference from default (package-private) is exactly the **subclass-in-another-package** case. **WellDev asked this in Round 1.**

B5. Correct order of restrictiveness (most → least) in Java:

- a) private → default → protected → public
- b) public → protected → default → private
- c) private → protected → default → public
- d) default → private → public → protected

Answer: (a). private (class only) → default (package) → protected (package + subclasses) → public (everywhere).

B6. An abstract class:

- a) Cannot have a constructor
- b) Cannot be instantiated directly
- c) Cannot have concrete methods
- d) Cannot have fields

Answer: (b). It **can** have constructors — they run via `super()` when a concrete subclass is created.

B7. Key difference between an abstract class and an interface (classical Java)?

- a) An interface can hold state, an abstract class can't
- b) A class can extend many abstract classes
- c) An abstract class can have constructors and instance state; a class can implement multiple interfaces
- d) They're identical

Answer: (c). Rule of thumb: abstract class for an *is-a* hierarchy with shared state; interface for a *can-do* capability. Note that modern Java interfaces do allow default and static methods, but still no instance state.

B8. Encapsulation is best demonstrated by:

- a) A subclass overriding a parent method
- b) Private fields exposed through public getters/setters
- c) Two methods with the same name
- d) An abstract base class

Answer: (b).

B9. Abstraction vs encapsulation:

- a) Abstraction hides implementation complexity; encapsulation hides and binds data
- b) They are synonyms
- c) Abstraction hides data; encapsulation hides behaviour
- d) Encapsulation only applies to interfaces

Answer: (a). Abstraction is a **design-level** concern (what is exposed); encapsulation is an **implementation-level** one (how state is protected).

B10. Which is compile-time (static) polymorphism?

- a) Method overriding
- b) Virtual functions
- c) Operator/method overloading
- d) Dynamic dispatch

Answer: (c). The other three are all runtime polymorphism described in different words.

B11. A constructor:

- a) Can be overridden
- b) Can be overloaded
- c) Must return void
- d) Must be public

Answer: (b). Constructors are not inherited, so they cannot be overridden. They have no return type at all (not even void), and they **can be private** — which is how Singleton works.

B12. The "diamond problem" arises from:

- a) Multiple interface implementation
- b) Multiple class inheritance
- c) Method overloading
- d) Static methods

Answer: (b). If D inherits from B and C, which both inherit from A, which version of a method does D get? This ambiguity is why Java and C# ban multiple class inheritance and allow multiple interfaces instead. C++ solves it with virtual inheritance.

B13. Good design favours:

- a) High coupling, low cohesion
- b) Low coupling, high cohesion
- c) High coupling, high cohesion
- d) Low coupling, low cohesion

Answer: (b). Modules should depend on each other as little as possible (low coupling) while each module's own contents belong tightly together (high cohesion).

B14. The L in SOLID (Liskov Substitution) says:

- a) A class should have one reason to change
- b) Subtypes must be substitutable for their base types without breaking correctness
- c) Depend on abstractions
- d) Prefer many small interfaces

Answer: (b). The classic violation: Square extends Rectangle, where setting the width silently changes the height.

B15. "Open for extension, closed for modification" is:

- a) SRP
- b) OCP
- c) ISP
- d) DIP

Answer: (b) Open/Closed Principle. Full SOLID: **S**ingle responsibility, **O**pen/closed, **L**iskov substitution, **I**nterface segregation, **D**ependency inversion.

B16. is-a vs has-a:

- a) is-a = composition, has-a = inheritance
- b) is-a = inheritance, has-a = composition
- c) Both mean inheritance
- d) Both mean composition

Answer: (b).

Section C — Design Patterns

12 questions

C1. GoF patterns fall into which three categories?

- a) Creational, Structural, Behavioural
- b) Simple, Complex, Hybrid
- c) Static, Dynamic, Hybrid
- d) Class, Object, Interface

Answer: (a). 23 patterns in total: creational (object creation), structural (composition), behavioural (communication).

C2. Singleton is a ____ pattern and requires:

- a) Structural; a public constructor
- b) Creational; a private constructor and a static accessor
- c) Behavioural; multiple instances
- d) Creational; an abstract base class

Answer: (b). Be ready for the thread-safety follow-up: eager static initialisation, double-checked locking with volatile, or an enum-based singleton in Java.

C3. Which is *not* a creational pattern?

- a) Builder
- b) Prototype
- c) Adapter
- d) Factory Method

Answer: (c) Adapter — it is structural. The five creational patterns: Singleton, Factory Method, Abstract Factory, Builder, Prototype.

C4. Factory Method vs Abstract Factory:

- a) Identical
- b) Factory Method creates one product via subclass choice; Abstract Factory creates families of related products
- c) Abstract Factory is behavioural
- d) Factory Method requires interfaces only

Answer: (b). Abstract Factory is essentially a factory of factories — think a UI toolkit producing a matching button, checkbox and scrollbar per platform.

C5. Which pattern converts one interface into another the client expects?

- a) Decorator
- b) Adapter
- c) Facade
- d) Observer

Answer: (b) Adapter. The wrapper that makes two incompatible interfaces work together — the "travel plug" of design patterns.

C6. Adding responsibilities to an object dynamically at runtime, without subclassing:

- a) Proxy
- b) Decorator
- c) Bridge
- d) Composite

Answer: (b) Decorator. Wrap and re-wrap: new BufferedReader(new FileReader(f)) is the canonical example from the Java I/O library.

C7. Decorator vs Proxy — the main distinction:

- a) Decorator adds behaviour; Proxy controls access
- b) Proxy adds behaviour; Decorator controls access
- c) No difference
- d) Proxy is creational

Answer: (a). Both wrap an object with the same interface, but intent differs: a Proxy handles lazy loading, access control, caching or remoting.

C8. A simplified single entry point over a complex subsystem:

- a) Facade
- b) Mediator
- c) Flyweight
- d) Bridge

Answer: (a) Facade. Structural. Mediator is behavioural and centralises communication *between* peer objects — the usual distractor.

C9. One-to-many dependency where dependents are auto-notified of a state change:

- a) Strategy
- b) Observer
- c) Command
- d) State

Answer: (b) Observer. Event listeners, pub/sub, and reactive streams are all this pattern.

C10. Strategy vs State:

- a) Strategy swaps interchangeable algorithms chosen by the client; State changes behaviour as internal state changes
- b) Identical
- c) State is creational
- d) Strategy is structural

Answer: (a). Structurally near-identical; the difference is who decides and why. In Strategy the client picks; in State the object transitions itself.

C11. Dependency Injection is:

- a) One of the 23 GoF patterns
- b) A form of Inversion of Control, realising the Dependency Inversion Principle
- c) A creational GoF pattern
- d) A database technique

Answer: (b). WellDev asked "what do you understand by Dependency Injection?" in Round 1. Note the trap: it is *not* a GoF pattern — the GoF book predates the term.

C12. Rails (WellDev's main backend stack) is built on which architectural and data patterns?

- a) MVVM + DAO
- b) MVC + Active Record
- c) MVP + Repository
- d) Layered + Data Mapper

Answer: (b). Active Record (Fowler) puts persistence logic on the domain object itself; Data Mapper (Hibernate, Doctrine) separates them. Rails also leans on **Convention over Configuration**. Worth knowing given their stack.

Section D — Operating Systems

16 questions

D1. Two threads of the same process do *not* share:

- a) Heap
- b) Code segment
- c) Stack and registers
- d) Global variables

Answer: (c). Each thread owns a private stack, register set and program counter. Everything else — code, heap, globals, open file descriptors — is shared.

D2. Which is true?

- a) Context switching between threads is costlier than between processes
- b) Threads are lightweight; thread switching is cheaper
- c) Each thread has its own address space
- d) Processes share the heap by default

Answer: (b). A thread switch keeps the same address space, so the page tables and TLB stay valid — a process switch has to flush them.

D3. The four necessary conditions for deadlock:

- a) Mutual exclusion, hold and wait, no preemption, circular wait
- b) Starvation, aging, priority, preemption
- c) Paging, swapping, thrashing, fragmentation
- d) Atomicity, isolation, locking, waiting

Answer: (a). All four must hold at once, so removing any single one prevents deadlock.

D4. Banker's algorithm is used for deadlock:

- a) Prevention
- b) Avoidance
- c) Detection
- d) Recovery

Answer: (b) avoidance. It requires processes to declare maximum resource needs in advance, then only grants a request if the system stays in a **safe state**.

D5. Which scheduling algorithm gives the minimum average waiting time?

- a) FCFS
- b) SJF
- c) Round Robin
- d) Priority

Answer: (b) Shortest Job First. Provably optimal for average waiting time, but it can **starve** long jobs and requires knowing burst times in advance, which is why it is rarely used as-is.

D6. Round Robin with a very large time quantum degenerates into:

- a) SJF
- b) FCFS
- c) Priority
- d) Multilevel queue

Answer: (b) FCFS. If the quantum exceeds every burst, no process is ever preempted. At the other extreme, a tiny quantum wastes everything on context switching.

D7. Mutex vs semaphore:

- a) Identical
- b) A mutex has ownership (only the locker unlocks); a semaphore is a signalling counter with no ownership
- c) A mutex can count beyond 1
- d) A semaphore is always binary

Answer: (b). Mutex = locking mechanism for mutual exclusion. Semaphore = signalling mechanism; a counting semaphore admits N concurrent holders and any thread may signal it.

D8. A race condition occurs when:

- a) Two processes run on different cores
- b) The result depends on the uncontrolled timing or order of concurrent accesses to shared data
- c) A process waits indefinitely
- d) Memory is exhausted

Answer: (b). The fix is to make the critical section atomic with a lock. Option (c) describes starvation or deadlock.

D9. Belady's anomaly (more frames → more page faults) is exhibited by:

- a) LRU
- b) Optimal
- c) FIFO
- d) All of them

Answer: (c) FIFO only. LRU and Optimal are **stack algorithms** — the pages held with n frames are always a subset of those held with n+1 — so they are immune. High-frequency exam item.

D10. Paging causes _____ fragmentation; segmentation causes _____ fragmentation.

- a) external, internal
- b) internal, external
- c) internal, internal
- d) external, external

Answer: (b) internal, external. Fixed-size pages waste space *inside* the last page (internal). Variable-size segments leave unusable gaps *between* allocations (external).

D11. Thrashing is:

- a) CPU overheating
- b) A process spending more time paging than executing
- c) Deadlock
- d) Excessive context switching between threads

Answer: (b). Caused by over-committing memory; the working-set model and page-fault-frequency control are the standard mitigations.

D12. The kernel is:

- a) A user application
- b) The core of the OS, managing CPU, memory and devices, and running in privileged mode
- c) The shell
- d) The bootloader

Answer: (b). Asked verbatim in their Round 1. The shell is just a user-space program that talks to the kernel.

D13. A system call causes a transition from:

- a) Kernel mode to user mode
- b) User mode to kernel mode
- c) Ready to running
- d) Running to terminated

Answer: (b). A software interrupt / trap raises the privilege level so the kernel can do the privileged work, then returns to user mode.

D14. In C, fork() returns:

- a) 0 in the parent, child PID in the child
- b) 0 in the child, child PID in the parent
- c) 1 in both
- d) The parent's PID in the child

Answer: (b). That asymmetry is how each side knows who it is. On failure `fork()` returns `-1` in the parent and no child is created.

D15. A zombie process is one that:

- a) Has terminated but whose exit status the parent hasn't reaped
- b) Has lost its parent
- c) Is stuck in an infinite loop
- d) Is waiting for I/O

Answer: (a). The parent clears it by calling `wait()`. Option (b) describes an **orphan**, which is adopted by `init/systemd`.

D16. Virtual memory allows:

- a) Faster CPU clock
- b) Programs larger than physical RAM to execute
- c) Deadlock prevention
- d) Multi-core scheduling

Answer: (b). It also gives each process an isolated address space, which is a security benefit as much as a capacity one.

E1. OSI layers in order (Layer 1 → 7):

- a) Physical, Data Link, Network, Transport, Session, Presentation, Application
- b) Physical, Network, Data Link, Transport, Application, Session, Presentation
- c) Application, Presentation, Session, Transport, Network, Data Link, Physical
- d) Physical, Data Link, Transport, Network, Session, Presentation, Application

Answer: (a). Mnemonic bottom-up: *Please Do Not Throw Sausage Pizza Away*. Note (c) is the same list top-down — read the direction the question asks for.

E2. A router operates at which layer?

- a) 1
- b) 2
- c) 3
- d) 4

Answer: (c) Layer 3. Switch = Layer 2 (MAC), hub = Layer 1 (electrical repeater).

E3. TCP's three-way handshake is:

- a) SYN → ACK → FIN
- b) SYN → SYN-ACK → ACK
- c) SYN → FIN → ACK
- d) ACK → SYN → SYN-ACK

Answer: (b). Termination is a **four-way** exchange: FIN, ACK, FIN, ACK — a common follow-up question.

E4. Which protocol suits live video calling?

- a) TCP, for guaranteed delivery
- b) UDP, because low latency beats retransmitting stale frames
- c) FTP
- d) SMTP

Answer: (b). A frame that arrives late is worthless, so retransmission is a cost with no benefit. Same reasoning for online gaming and DNS.

E5. Which is *not* a TCP feature?

- a) Connection-oriented
- b) Flow control
- c) Guaranteed ordering
- d) Lowest possible overhead

Answer: (d). TCP's reliability guarantees cost header bytes, handshakes and acknowledgements. Minimal overhead is UDP's selling point.

E6. DNS primarily uses:

- a) TCP port 53
- b) UDP port 53
- c) UDP port 80
- d) TCP port 25

Answer: (b) UDP/53. TCP/53 is used for zone transfers and responses too large for a single datagram.

E7. Match the default ports: SSH, HTTPS, SMTP, FTP control

- a) 22, 443, 25, 21
- b) 23, 80, 110, 20
- c) 22, 80, 25, 21
- d) 21, 443, 143, 22

Answer: (a). Also memorise: HTTP 80, Telnet 23, DNS 53, POP3 110, IMAP 143, MySQL 3306, PostgreSQL 5432. FTP uses 21 for control and 20 for data.

E8. For 192.168.10.130/26, the network address is:

- a) 192.168.10.0
- b) 192.168.10.64
- c) 192.168.10.128
- d) 192.168.10.192

Answer: (c) 192.168.10.128. /26 → mask 255.255.255.192, block size 64, so subnets start at 0 / 64 / 128 / 192. 130 lands in the fourth-from-last block: network **.128**, broadcast **.191**, 62 usable hosts. **Their Round 1 list includes "finding the network address" — practise several of these.**

E9. Which is a private IP range?

- a) 11.0.0.0/8
- b) 172.16.0.0/12
- c) 192.169.0.0/16
- d) 8.8.8.0/24

Answer: (b). The three private ranges are **10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16**. Note the near-misses: 11.x and 192.169.x are public, and 8.8.8.8 is Google's public DNS.

E10. ARP resolves:

- a) MAC → IP
- b) IP → MAC
- c) Domain → IP
- d) IP → Domain

Answer: (b) IP → MAC. RARP is the reverse; DNS is domain → IP; reverse DNS is IP → domain.

E11. HTTP status class meanings — which mapping is right?

- a) 3xx client error, 4xx redirect
- b) 4xx client error, 5xx server error
- c) 2xx redirect, 3xx success
- d) 5xx client error, 4xx server error

Answer: (b). 1xx informational, 2xx success, 3xx redirection, 4xx client error, 5xx server error.

E12. A successful POST creating a resource should return:

- a) 200
- b) 201
- c) 204
- d) 302

Answer: (b) 201 Created. 204 is success *with no body*, typical for DELETE.

E13. 401 vs 403:

- a) 401 = not authenticated; 403 = authenticated but not authorised
- b) The reverse
- c) Both mean "login failed"
- d) 403 means resource missing

Answer: (a). Confusingly, 401's official name is "Unauthorized" even though it is really about *authentication*.

E14. Which HTTP methods are idempotent?

- a) GET, PUT, DELETE
- b) POST, PATCH
- c) POST only
- d) None

Answer: (a). Safe methods (no side effects): GET, HEAD, OPTIONS. Idempotent: those three plus PUT and DELETE. **POST is neither safe nor idempotent**, and PATCH is generally not idempotent either.

E15. PUT vs PATCH:

- a) PUT partially updates; PATCH replaces
- b) PUT replaces the full resource; PATCH applies a partial update
- c) Identical
- d) PATCH is always idempotent

Answer: (b). Option (a) reverses them and (d) is the subtle trap — a PATCH such as "add 1 to the counter" is not idempotent.

E16. REST is stateless, meaning:

- a) The server stores no session for the client between requests; each request carries all needed context
- b) No data is persisted
- c) Responses can't be cached
- d) Only GET is allowed

Answer: (a). Option (b) is the classic misreading: the **database** still stores data; it is the per-client **session** that the server does not keep. This is what allows any server behind a load balancer to answer any request.

E17. A browser on localhost:3000 calling an API on localhost:8000 is blocked. This is:

- a) A firewall issue
- b) CORS — fixed server-side via Access-Control-Allow-Origin
- c) A DNS failure
- d) A client-side JavaScript bug

Answer: (b). A different **port** is a different origin under the same-origin policy. The browser enforces it, but only the **server** can relax it with the right headers — non-simple requests also trigger an OPTIONS preflight.

E18. HTTPS = HTTP over:

- a) SSH
- b) TLS/SSL
- c) UDP
- d) IPSec

Answer: (b) TLS/SSL. Default port 443. It provides encryption, integrity and server authentication via certificates.

Section F — Data Structures and Algorithms

26 questions

F1. Time complexity of arr[2] on an array, and the same index access on a singly linked list:

- a) O(1), O(1)
- b) O(1), O(n)
- c) O(n), O(1)
- d) O(n), O(n)

Answer: (b). Arrays are contiguous, so address = base + i × size. **Their Round 1 asked this exact pair.**

F2. Printing an array name in C, printf("%p", arr), yields:

- a) The first element's value
- b) The base address (address of arr[0])
- c) The array size
- d) Garbage

Answer: (b). &arr[0] and &arr hold the same address but have different types (int* vs int(*)[3]), so they behave differently under pointer arithmetic. **Asked in their Round 1.**

F3. Binary search on a sorted singly linked list is:

- a) O(log n), same as arrays
- b) O(n) overall, because reaching the middle requires traversal
- c) Impossible
- d) O(1)

Answer: (b). Note that (c) is wrong — it is possible, just pointless. Without random access the logarithmic benefit collapses.

F4. Which sorting algorithms are stable?

- a) Quick, Heap, Selection
- b) Merge, Insertion, Bubble
- c) Heap, Merge, Quick
- d) Selection, Heap, Quick

Answer: (b). Stable = equal keys keep their original relative order. Quick, heap and selection sort are **unstable**. Counting and radix sort are also stable.

F5. Merge sort's auxiliary space complexity:

- a) O(1)
- b) O(log n)
- c) O(n)
- d) O(n log n)

Answer: (c) O(n). The merge step needs a temporary buffer. This is the main reason quicksort (in-place, O(log n) stack) is often preferred in practice despite a worse worst case.

F6. Quicksort's worst case is $O(n^2)$ when:

- a) The array is random
- b) The pivot is consistently the smallest or largest element
- c) The array has duplicates
- d) n is a power of 2

Answer: (b). With a naive "first element" pivot, already-sorted input triggers exactly this. Randomised or median-of-three pivot selection fixes it in practice.

F7. Best algorithm for a nearly sorted array:

- a) Selection sort
- b) Insertion sort
- c) Heap sort
- d) Quick sort

Answer: (b) insertion sort. $O(n)$ best case — it is adaptive, doing almost no work when elements are already near their final positions.

F8. Selection sort vs insertion sort:

- a) Selection is $O(n^2)$ in all cases; insertion is $O(n)$ best case
- b) Both are $O(n)$ best case
- c) Selection is stable, insertion isn't
- d) Insertion does fewer swaps

Answer: (a). Selection sort does the **fewest swaps** ($n-1$), insertion sort the fewest comparisons on nearly sorted data. Note (c) is backwards: insertion is the stable one. **Asked in their Round 1.**

F9. Which sort guarantees $O(n \log n)$ worst case and $O(1)$ extra space?

- a) Merge sort
- b) Quick sort
- c) Heap sort
- d) Insertion sort

Answer: (c) heap sort. Merge sort has the time guarantee but needs $O(n)$ space; quicksort is in-place but has an $O(n^2)$ worst case. Heap sort's trade-off is poor cache locality and instability.

F10. Which BST traversal yields keys in descending order?

- a) Inorder (L-Root-R)
- b) Reverse inorder (R-Root-L)
- c) Preorder
- d) Postorder

Answer: (b). Round 1 asked how to iterate a BST in reverse order. Inorder gives ascending; mirroring it gives descending.

F11. Search in a BST costs:

- a) Always $O(\log n)$
- b) $O(h)$, where h is the height — $O(\log n)$ balanced, $O(n)$ degenerate
- c) $O(1)$
- d) $O(n \log n)$

Answer: (b). Option (a) is the trap: inserting sorted data into a plain BST produces a linked list. Self-balancing trees (AVL, red-black) restore the $O(\log n)$ guarantee.

F12. A perfect binary tree of height h (root at height 0) has how many nodes?

- a) 2^h
- b) $2^{(h+1)} - 1$
- c) $2h + 1$
- d) h^2

Answer: (b). Leaves = 2^h , and the leaves make up just over half of all nodes.

F13. In any binary tree, the number of nodes with two children equals:

- a) leaves
- b) leaves $- 1$
- c) leaves $+ 1$
- d) internal nodes

Answer: (b) leaves $- 1$. Each two-child node adds exactly one extra leaf beyond the first, so the relation holds for every binary tree.

F14. DFS and BFS use which data structures respectively?

- a) Queue, Stack
- b) Stack, Queue
- c) Heap, Stack
- d) Queue, Queue

Answer: (b). Watch the order the question asks for — (a) is the same pair reversed. **Asked in their Round 1.**

F15. Building a heap from an unsorted array of n elements takes:

- a) $O(n \log n)$
- b) $O(n)$
- c) $O(\log n)$
- d) $O(n^2)$

Answer: (b) $O(n)$. Bottom-up heapify is linear because most nodes are near the bottom and sift down only a short distance. Inserting the n elements one at a time is $O(n \log n)$ — that is the distractor.

F16. Extract-min from a min-heap, and peek-min:

- a) $O(\log n)$, $O(1)$
- b) $O(1)$, $O(\log n)$
- c) $O(n)$, $O(1)$
- d) $O(\log n)$, $O(\log n)$

Answer: (a). Peek just reads the root; extract must move the last element to the root and sift it down.

F17. An MST of a connected graph with V vertices has exactly:

- a) V edges
- b) $V - 1$ edges
- c) $E - V$ edges
- d) $2V$ edges

Answer: (b) $V - 1$. True of any spanning tree — one more edge would create a cycle, one fewer would disconnect it.

F18. Kruskal's algorithm is:

- a) Dynamic programming, using a matrix
- b) Greedy: sort edges by weight and use union-find to reject cycles
- c) Divide and conquer
- d) Backtracking

Answer: (b). $O(E \log E)$, dominated by the sort. Prim's is greedy too, at $O(E \log V)$ with a binary heap.

F19. Prim's vs Kruskal's — Prim's is generally preferred for:

- a) Sparse graphs
- b) Dense graphs
- c) Directed graphs
- d) Weighted graphs only

Answer: (b) dense graphs. Prim's grows one tree from a vertex and scales with V ; Kruskal's sorts all E edges, which suits **sparse** graphs. (Neither applies to directed graphs — that needs an arborescence algorithm.)

F20. Dijkstra's algorithm fails when:

- a) The graph is dense
- b) There are negative edge weights
- c) The graph is undirected
- d) There are more than 1000 nodes

Answer: (b). Once a node is finalised Dijkstra never revisits it, which a negative edge can invalidate. Use **Bellman-Ford**, $O(V \cdot E)$, which also detects negative cycles.

F21. Which problem is solved optimally by a greedy approach?

- a) 0/1 Knapsack
- b) Fractional Knapsack
- c) Longest Common Subsequence
- d) Matrix Chain Multiplication

Answer: (b) Fractional Knapsack. Take items by value-to-weight ratio and split the last one. **0/1 Knapsack needs DP** because you cannot break an item. Activity Selection and Huffman coding are the other standard greedy wins.

F22. Complexity of:

```
int fun(int n) {  
    if (n <= 1) return n;  
    return fun(n-1) + fun(n-2);  
}
```

- a) $O(n)$
- b) $O(n \log n)$
- c) $O(2^n)$ — exponential
- d) $O(n^2)$

Answer: (c). Naïve Fibonacci — each call spawns two more and recomputes the same subproblems. **Memoisation drops it to $O(n)$. Asked in their Round 1.**

F23. For $f(n) = n + f(n-1)$, the recursive version vs an iterative loop:

- a) Same time $O(n)$, same space
- b) Same time $O(n)$; recursion uses $O(n)$ stack space, iteration $O(1)$
- c) Recursion is faster
- d) Iteration uses more space

Answer: (b). Round 1 asked exactly this comparison. The headline is that time is identical and only *space* differs — plus the risk of stack overflow at large n .

F24. Best data structure for prefix-based string search / autocomplete:

- a) Array
- b) Trie
- c) Stack
- d) Min-heap

Answer: (b) trie. Lookup is $O(L)$ in key length, independent of dictionary size. **Asked in their Round 1.**

F25. Hash table lookup: average and worst case?

- a) $O(1)$, $O(1)$
- b) $O(1)$, $O(n)$
- c) $O(\log n)$, $O(n)$
- d) $O(n)$, $O(n)$

Answer: (b). The worst case occurs when every key collides into one bucket. Some implementations mitigate it by converting long chains into balanced trees, giving $O(\log n)$.

F26. Adjacency matrix vs adjacency list for a graph:

- a) Matrix $O(V+E)$ space, list $O(V^2)$
- b) Matrix $O(V^2)$ space with $O(1)$ edge lookup; list $O(V+E)$ space, better for sparse graphs
- c) Identical
- d) Lists can't store weights

Answer: (b). Matrix wins on "is there an edge $u-v$?" in $O(1)$; list wins on space and on iterating a vertex's neighbours. Most real graphs are sparse, so lists are the usual default.

Item	Value
SQL execution order	FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT
Normal forms	1NF atomic · 2NF no partial dependency · 3NF no transitive dependency · BCNF every determinant is a superkey
ACID	Atomicity all-or-nothing · Consistency valid → valid · Isolation no interference · Durability survives a crash
DELETE / TRUNCATE / DROP	DELETE = DML, WHERE, triggers, rollback · TRUNCATE = DDL, all rows, resets auto-increment · DROP = removes the table
Belady's anomaly	FIFO only (LRU and Optimal are stack algorithms and immune)
Deadlock conditions	Mutual exclusion · hold and wait · no preemption · circular wait
Fragmentation	Paging → internal · Segmentation → external
Threads share / don't share	Share: code, heap, globals, file descriptors. Private: stack, registers, program counter
Stable sorts	Insertion, Merge, Bubble, Counting, Radix
O(n log n) guaranteed	Merge (O(n) space), Heap (O(1) space). Quicksort is O(n log n) average, O(n²) worst
Best case O(n)	Insertion sort and bubble sort (adaptive). Selection sort is O(n²) always
Build-heap	O(n) bottom-up · O(n log n) if inserted one by one
MST edges	V − 1 · Kruskal O(E log E) sparse · Prim O(E log V) dense
Shortest path	Dijkstra (no negative weights) · Bellman-Ford O(V·E), handles negatives and detects negative cycles
DFS / BFS	DFS = stack · BFS = queue (and BFS gives shortest path in an unweighted graph)
BST traversals	Inorder L-Root-R = ascending · Reverse inorder R-Root-L = descending
Safe HTTP methods	GET, HEAD, OPTIONS
Idempotent HTTP	GET, HEAD, OPTIONS, PUT, DELETE (POST and PATCH are not)
Status codes	201 created · 204 no content · 400 bad request · 401 not authenticated · 403 forbidden · 404 not found · 409 conflict · 429 rate limited · 500 server error
OSI 1→7	Physical, Data Link, Network, Transport, Session, Presentation, Application — <i>Please Do Not Throw Sausage Pizza Away</i>
Default ports	FTP 20/21 · SSH 22 · Telnet 23 · SMTP 25 · DNS 53 · HTTP 80 · POP3 110 · IMAP 143 · HTTPS 443 · MySQL 3306 · PostgreSQL 5432
Private IP ranges	10.0.0.0/8 · 172.16.0.0/12 · 192.168.0.0/16
/26 subnet	Mask 255.255.255.192, block size 64, 62 usable hosts, boundaries 0 / 64 / 128 / 192
Usable hosts formula	2 ^(32 − prefix) − 2 → /24 = 254, /25 = 126, /26 = 62, /27 = 30, /28 = 14
JS event loop	Synchronous code → microtasks (promises) → macrotasks (setTimeout)
SOLID	Single responsibility · Open/closed · Liskov substitution · Interface segregation · Dependency inversion
GoF categories	Creational (Singleton, Factory Method, Abstract Factory, Builder, Prototype) · Structural (Adapter, Decorator, Facade, Proxy, Bridge, Composite, Flyweight) · Behavioural (Observer, Strategy, State, Command, ...)

1. Array address: &arr[0] versus &arr

Q2, F2

&arr[0] points to the first integer; &arr points to the whole array. The numerical starting addresses are the same, but the types and therefore the pointer arithmetic differ:

```
&arr[0] + 1    // moves by sizeof(int)    -> 4 bytes
&arr    + 1    // moves by sizeof(arr)    -> 12 bytes for int arr[3]
```

2. Array versus linked-list access

Q4, Q5, F1

An array supports direct address calculation, so element access is independent of the index:

```
address = base + index × element_size    -> arr[index] is O(1)
```

A linked list stores nodes anywhere in memory, connected only by pointers, so reaching the k-th element means following k links — O(k). The trade-off runs the other way for insertion at the head: O(n) for an array (everything shifts), O(1) for a linked list.

3. Binary search on a linked list

Q33, F3

It is logically possible to apply binary-search comparisons to a sorted linked list — the trap answer is “impossible.” But locating each midpoint takes a linear traversal, so the total degenerates to O(n) and the logarithmic advantage disappears. Binary search needs **sorted data plus random access**.

4. Recursive Fibonacci

Q43, F22

The function recomputes the same values over and over:

```
fib(5)
├─ fib(4)
│   └─ fib(3)
│       └─ fib(2)
└─ fib(3)    <- already computed inside fib(4)
```

The call tree roughly doubles at each level, giving O(2ⁿ) time and O(n) stack depth. Memoisation (top-down) or a bottom-up DP loop reduces it to **O(n) time**, and the bottom-up version needs only O(1) space if you keep two variables.

5. SQL execution order

Q49, A9, A10

```
FROM / JOIN
→ WHERE
→ GROUP BY
→ HAVING
→ SELECT
→ DISTINCT
→ ORDER BY
→ LIMIT / OFFSET
```

Two consequences examiners love: a SELECT alias is unusable in WHERE (WHERE runs first) but usable in ORDER BY; and aggregate functions belong in HAVING, never in WHERE.

6. Second-highest salary

Q60, A20

The word **distinct** matters. If the two highest-paid employees earn the same amount, the expected answer is the second-highest *distinct* salary:

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- window-function alternative
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t WHERE rnk = 2;
```

The subquery form returns NULL when every salary is equal — a good detail to mention aloud in an interview.

7. The /26 subnet

Q81, Q82, E8

A /26 network leaves six host bits:

```
26 = 64 total addresses
64 – network address – broadcast address = 62 usable hosts
mask = 255.255.255.192, block size = 64
```

Subnet ranges in the last octet are **0-63, 64-127, 128-191, 192-255**. So 192.168.1.**70**/26 sits in the second block — network 192.168.1.64, broadcast 192.168.1.127. Method for any prefix: block size = 256 – the interesting octet of the mask, then find which multiple of the block size your address falls into.

8. The two-egg problem

Q95

With a fixed step size of k you need up to $100/k + k$ drops. Instead use a **decreasing** step so every path costs the same total. If the first drop is at floor n , the next jump is $n-1$ floors, then $n-2$, and so on:

$$\begin{aligned} n + (n-1) + (n-2) + \dots + 1 &= n(n+1)/2 \geq 100 \\ n = 14 \rightarrow 14 \times 15 / 2 = 105 \quad \checkmark \qquad n = 13 \rightarrow 91 \quad \times \end{aligned}$$

So drop from floor 14, then 27, 39, 50, 60, 69, 77, 84, 90, 95, 99, 100. Whenever the first egg breaks, walk the remaining interval one floor at a time with the second egg; the worst case is always **14** drops.

Tick these off the night before. If any line makes you hesitate, that is where your remaining study time goes.

Data structures and algorithms	Highest weight — 22%
• Array versus linked-list complexity, in both directions (access and head insertion) • Stack, queue, BFS and DFS — which structure each uses, and why • Binary search: requirements, complexity, and why it fails on a linked list • BST traversal orders, including reverse inorder for descending output • Merge, quick, insertion and selection sort — best/average/worst time, space, stability • Naïve recursion versus dynamic programming, and recursion's O(n) stack cost • Hash tables, heaps, tries, and MST algorithms (Kruskal, Prim) • Dijkstra versus Bellman-Ford, and when greedy is provably optimal	
OOP and design patterns	14%
• Encapsulation, abstraction, inheritance, polymorphism — one clean sentence each • Overloading versus overriding, and compile-time versus runtime binding • Abstract class versus interface; access modifiers and their exact scope • Singleton, Builder, Strategy, Observer, Adapter, Decorator — and Decorator versus Proxy • SOLID, plus Dependency Injection as Inversion of Control (not a GoF pattern)	
DBMS and SQL	16%
• SQL execution order and its two consequences (aliases, aggregates) • Joins, GROUP BY, WHERE versus HAVING • Normalization 1NF through BCNF, plus why you would denormalize • ACID properties and isolation levels (dirty, non-repeatable and phantom reads) • Duplicate-row and second-highest-salary queries, written from memory • DELETE, TRUNCATE and DROP; indexes and their write cost	
API, HTTP, networking and OS	28% combined
• REST statelessness; safe versus idempotent methods; PUT versus PATCH • Status codes, especially 201, 204, 400, 401, 403, 404, 409 and 429 • CORS: why it happens and where it is fixed • Subnet masks, /24 and /26, network address and usable host counts • TCP versus UDP, the three-way handshake, DNS, ARP, OSI layers and default ports • Process versus thread; what threads do and do not share • The four deadlock conditions; scheduling basics; virtual memory and thrashing • Password hashing with a unique salt; SQL-injection prevention with prepared statements • JavaScript scope, == versus ===, promises and event-loop ordering	
Exam-day tactics. Answer every question you know first and flag the rest — MCQ rounds are usually time-pressured, and an easy question left blank at the end costs the same as a hard one you got wrong. Watch for reversed option pairs (“Stack, Queue” versus “Queue, Stack”) and for absolute words like <i>always</i>, <i>never</i> and <i>only</i>, which usually mark a false option. Where a question offers a plausible answer and a stricter one — O(log n) versus O(h) for BST search, for example — the stricter one is normally intended.	

Paper 1 — Questions 1-100														By section
Section	Answers													
A • Programming & JavaScript	1-B	2-A	3-B	4-A	5-C	6-C	7-A	8-B	9-C	10-C	11-B	12-C		
B • OOP & design patterns	13-B	14-B	15-C	16-A	17-B	18-B	19-C	20-C	21-B	22-B	23-B	24-B	25-A	
C • DSA	27-B	28-A	29-B	30-C	31-A	32-B	33-C	34-B	35-B	36-C	37-B	38-B	39-C	
	40-D	41-C	42-D	43-D	44-A	45-B	46-A	47-B	48-B					
D • DBMS & SQL	49-B	50-B	51-C	52-B	53-B	54-B	55-A	56-B	57-C	58-B	59-B	60-B	61-C	
	62-B	63-B	64-C											
E • API & HTTP	65-B	66-A	67-A	68-B	69-B	70-B	71-C	72-A	73-C	74-C	75-B			
F • Networking	76-B	77-C	78-B	79-B	80-B	81-C	82-B	83-B						
G • OS, security & SWE	84-B	85-B	86-A	87-A	88-B	89-B	90-B	91-B	92-A					
H • Analytical reasoning	93-C	94-C	95-C	96-C	97-C	98-B	99-B	100-C						

Paper 2 — 110 questions														By section
Section	Answers													
A • DBMS & normalization	A1-b	A2-b	A3-b	A4-b	A5-b	A6-b	A7-b	A8-c	A9-b	A10-b	A11-c	A12-d		
	A13-d	A14-a	A15-b	A16-b	A17-b	A18-b	A19-b	A20-a	A21-a	A22-b				
B • OOP	B1-a	B2-b	B3-b	B4-c	B5-a	B6-b	B7-c	B8-b	B9-a	B10-c	B11-b	B12-b		
	B13-b	B14-b	B15-b	B16-b										
C • Design patterns	C1-a	C2-b	C3-c	C4-b	C5-b	C6-b	C7-a	C8-a	C9-b	C10-a	C11-b	C12-b		
D • Operating systems	D1-c	D2-b	D3-a	D4-b	D5-b	D6-b	D7-b	D8-b	D9-c	D10-b	D11-b	D12-b		
	D13-b	D14-b	D15-a	D16-b										
E • Networking & HTTP	E1-a	E2-c	E3-b	E4-b	E5-d	E6-b	E7-a	E8-c	E9-b	E10-b	E11-b	E12-b		
	E13-a	E14-a	E15-b	E16-a	E17-b	E18-b								
F • DSA	F1-b	F2-b	F3-b	F4-b	F5-c	F6-b	F7-b	F8-a	F9-c	F10-b	F11-b	F12-b		
	F13-b	F14-b	F15-b	F16-a	F17-b	F18-b	F19-b	F20-b	F21-b	F22-c	F23-b			
	F24-b	F25-b	F26-b											

A note on Paper 2's answer distribution. Notice how many answers are (b). That is an artefact of how this bank was written, not a pattern you will find in a real paper — do not let it train you to guess (b) under time pressure.